School_Management_System 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: Laravel/MySQL 기반 학교 관리 시스템을 Next.js 16 + Supabase로 마이그레이션하는 프로젝트. 관리자(교사)의 학급/과목/학생 CRUD와 실시간 출석 체크, 학생의 본인 출석 조회가 목표.
- 목적/대상 사용자: 관리자(admin, 교사 포함)와 학생(student) 2개 역할.

구분	기술
프론트	Next.js ^16.2.9 (App Router, src/app), React 19.2, Tailwind CSS v4, 아이콘은 이모지 문자
백엔드	별도 서버 없음. 서버 컴포넌트에서 Supabase 직접 조회. 서버 액션 파일은 작업 트리에서 삭제됨
DB	Supabase PostgreSQL (profiles, classes, subjects, attendance). 스키마/RLS SQL은 CLAUDE.md 일부 예시만 존재, 마이그레이션 폴더 없음
인프라	로컬 개발. .gitignore 삭제 상태, 상위 platform 저장소와 nested .git 이중 추적
AI	없음

- 현재 완성도 추정: 작업 트리 기준 약 25% (nested git HEAD 기준 약 55%)
  - 근거: 작업 트리에는 로그인, 관리자 레이아웃, 관리자 대시보드, 실시간 피드만 남아 있다. nested 저장소의 마지막 커밋(3c19d1e)에는 admin/{attendance,classes,students,subjects,settings}, student/{layout,dashboard}, lib/actions.ts, components/admin/* 10개 파일이 존재하지만 작업 트리에서 모두 삭제되어 있다(git -C School_Management_System status). 사이드바가 가리키는 /admin/classes, /admin/subjects, /admin/attendance는 404이고, 학생 역할은 로그인 후 갈 곳이 없다. 미들웨어는 위치 문제로 실행되지 않는다(3절 #2).
- 핵심 기능 목록(작업 트리 기준)
  - 이메일/구글 로그인, OAuth 콜백 후 역할별 리다이렉트
  - 관리자 대시보드: 전체 학생 수, 금일 출석률, 결석자 수, 학급별 출석률 막대, 최근 출석 변경 실시간 피드
  - 공통 UI: Card, Badge, Toast, Sidebar
  - 학생 실시간 알림 컴포넌트(RealtimeStudentNotif)는 존재하나 사용처 없음

2. 구조 분석

경로	역할
middleware.ts (프로젝트 루트)	세션 갱신, 비로그인 -> /login, 역할별 /admin·/student 차단. src/ **바깥에 있어 Next.js가 로드하지 않음**
src/lib/supabase/client.ts, server.ts	@supabase/ssr 브라우저/서버 클라이언트. Database 제네릭 없음
src/lib/types.ts	Role, AttendanceStatus(한글 리터럴), Class/Subject/Profile/Attendance 인터페이스(수기)
src/app/auth/callback/route.ts	OAuth code 교환 후 profiles.role 조회 -> 대시보드 리다이렉트
src/app/login/page.tsx	이메일 로그인 + 구글 OAuth 버튼, Toast 피드백
src/app/admin/layout.tsx	서버에서 profile(name, email) 조회 -> Sidebar + 헤더. 역할 검사 없음, 로그아웃 없음
src/app/admin/dashboard/page.tsx	force-dynamic. 통계 3종, 학급별 출석률, 최근 피드 초기 데이터 조회
src/components/realtime/RealtimeFeed.tsx	attendance 테이블 변경 구독 -> 최근 10건 재조회

경로	역할
src/components/realtime/RealtimeStudentNotif.tsx	특정 학생의 attendance UPDATE 구독 -> 5초 알림 (미사용)
src/components/ui/*	Card, Badge(미사용), Sidebar(4개 메뉴, 3개는 404), Toast
CLAUDE.md, PRD.md, Design_UI.md, School_Management_System.pen	가이드/기획/디자인
(삭제됨) src/lib/actions.ts, src/app/admin/{attendance,classes,students,subjects,settings}, src/app/student/(전체), src/components/admin/(전체), .gitignore	nested git HEAD에는 존재. 학급/과목/학생 CRUD 서버 액션, 출석 체크 보드(upsert + Realtime), 학생 대시보드

데이터 흐름: 의도된 흐름은 middleware.ts가 세션과 역할을 확인 -> admin/layout.tsx가 프로필을 읽어 레이아웃 렌더 -> 각 페이지가 서버 컴포넌트에서 조회하고 클라이언트 컴포넌트가 Realtime으로 갱신하는 구조다. 그러나 현재는 미들웨어가 실행되지 않으므로 admin/layout.tsx의 getUser()가 유일한 방어선이고, 그것도 user가 없을 때 redirect하지 않는다(user?.id ?? ''로 빈 조회만 한다). 출석 쓰기 경로(upsert)와 CRUD 서버 액션은 삭제된 파일에만 있어 작업 트리에는 쓰기 기능이 전혀 없다.

3. 발견된 문제점

#	심각도	분류	위치	내용
1	높음	구조	git status (nested repo): D src/lib/actions.ts, D src/app/admin/{attendance,classes,students,subjects,settings}/page.tsx, D src/app/student/{layout,dashboard}, D src/components/admin/*, D .gitignore	커밋된 기능 10개 파일이 작업 트리에서 삭제됨. src/components/ui/Sidebar.tsx:6-9의 메뉴 3개와 middleware.ts:37,59,65의 /student/dashboard 리다이렉트가 모두 404로 이어짐
2	높음	보안	middleware.ts:1-76 (루트 위치), tsconfig.json:345 (@/ -> ./src/)	앱이 src/app에 있으면 Next.js는 src/middleware.ts(16에서는 src/proxy.ts)만 인식함. 루트 middleware.ts는 로드되지 않아 인증·역할 가드가 전혀 실행되지 않음. CLAUDE.md:43도 src/lib/supabase/middleware.ts를 명시
3	높음	보안	CLAUDE.md:80-87 (유일한 RLS 정의), 저장소 전체 (supabase/ 폴더 없음)	admin 정책이 profiles 안에서 profiles를 서브쿼리로 참조 -> PostgreSQL "infinite recursion detected in policy" 오류 유발 패턴. classes/subjects/attendance 정책과 가입 트리거 SQL은 저장소에 없어 PRD F-AUTH-04 검증 불가
4	높음	설정	.gitignore (삭제됨, HEAD에는 .env*.local 포함), 상위 저장소 git ls-files School_Management_System 26개 파일	.gitignore가 없어 .env.local, .next/, node_modules/가 다음 git add -A에 포함될 위험. 상위 platform 저장소가 소스와 94KB .pen 파일을 중복 추적 중
5	중간	보안	src/app/admin/layout.tsx:5-11	레이아웃이 role을 확인하지 않고 user가 없어도 redirect하지 않음. 미들웨어가 꺼진 현재 상태에서는 어떤 로그인 사용자(학생 포함)도 /admin/dashboard 전체 통계 열람 가능
6	중간	버그	middleware.ts:55,64-68	profile이 없으면 role === undefined. /admin은 /student/dashboard로, /student는 /admin/dashboard로 보내 무한 리다이렉트 루프. 가입 트리거가 없으므로 신규 구글 가입자가 정확히 이 상태가 됨
7	중간	버그	src/app/admin/layout.tsx, src/components/ui/Sidebar.tsx	로그아웃 버튼/signOut 호출이 작업 트리 어디에도 없음(grep signOut 0건). 삭제된 settings 페이지에 있었던 것으로 추정
8	중간	버그	src/app/admin/dashboard/page.tsx:9,14	toISOString().split('T')[0]은 UTC 날짜 -> KST 00:00~09:00에 "오늘" 출석이 전날 기준으로 조회됨

#	심각도	분류	위치	내용
9	중간	버그	src/app/admin/dashboard/page.tsx:19-21,26-27	전체 출석률은 "출석 수 / 전체 학생 수", 학급별 출석률은 "출석 수 / 기록 수"로 분모가 다름. 과목이 여러 개면 전체 출석률이 100%를 초과할 수 있음
10	중간	품질	src/app/admin/dashboard/page.tsx:38, src/components/realtime/RealtimeFeed.tsx:35	(r: any) 명시적 any. next/typescript 규칙에서 린트 오류. PRD 5 "엄격한 타입 정의" 위반. join 결과 타입이 정의되지 않음
11	중간	설정	package.json:10,16-17,21	next lint는 Next 16에서 제거됨. typescript ^6.0.3, @types/node ^25.9.3는 생태계 호환 검증이 필요한 메이저. "main": "index.js"는 Next 프로젝트에 무의미
12	중간	구조	src/components/realtime/RealtimeStudentNotif.tsx, src/components/ui/Badge.tsx	사용처가 삭제되어 dead code. PRD F-DASH-02 학생 대시보드 자체가 없음
13	낮음	성능	src/components/realtime/RealtimeFeed.tsx:16,21-32	렌더마다 createClient() 호출(싱글턴이라 동작은 하나 의도 불명확), 필터 없는 전체 테이블 구독 + 이벤트마다 10건 재조회
14	낮음	품질	src/app/login/page.tsx:25,29-34	로그인 성공 시 window.location.href = '/' 전체 리로드. 구글 로그인 리다이렉트 지연 중 loading이 영구 true
15	낮음	품질	src/app/admin/dashboard/page.tsx:59,92, src/components/ui/Sidebar.tsx:16	grid-cols-3, w-60 고정. PRD 5 반응형 요구 미반영. 예외 처리/재시도(Exponential Backoff) 없음
16	낮음	품질	src/components/ui/Sidebar.tsx:6-9	이모지 아이콘 사용. Design_UI.md가 지정한 Indigo/Cyan 토큰은 적용되었으나 아이콘 체계 부재
17	낮음	설정	middleware.ts 파일명	Next 16에서 middleware는 deprecated, proxy.ts로 전환 권고. Hospital 프로젝트는 이미 src/proxy.ts 사용

심각도 높음 상세

#1 커밋된 기능의 작업 트리 삭제. nested 저장소 HEAD(3c19d1e)에는 학급/과목 CRUD 서버 액션(src/lib/actions.ts 67줄), 실시간 출석 체크 보드(src/app/admin/attendance/page.tsx 192줄, upsert ... onConflict: 'student_id,subject_id,date' + Realtime 구독), 학생 대시보드(99줄) 등 PRD 4.3~4.4의 핵심이 구현되어 있다. 작업 트리에서는 이 10개 파일이 삭제(D)되고 middleware.ts, dashboard/page.tsx, layout.tsx, login/page.tsx, Sidebar.tsx, types.ts, tsconfig.json만 수정(M)된 상태다. 의도적 롤백인지 실수인지 저장소에 기록이 없다. 어느 쪽이든 현재 트리는 사이드바 링크와 미들웨어 리다이렉트가 존재하지 않는 경로를 가리키는 깨진 상태이므로, 먼저 git stash 또는 git checkout -- <files>로 HEAD 기능을 복원하고 작업 트리 변경(미들웨어 개선 등)을 그 위에 재적용해야 한다.

#2 미들웨어가 실행되지 않는 위치. Next.js는 app/이 src/ 안에 있으면 미들웨어(16에서는 proxy)도 src/ 안에서 찾는다. 루트 middleware.ts는 빌드 시 조용히 무시되어 next build 출력에 "Middleware" 항목이 나오지 않는다. 결과적으로 (a) 비로그인 사용자가 /admin/dashboard에 접근하면 admin/layout.tsx가 user 없이 렌더를 시도하고, (b) 학생이 관리자 경로에 접근해도 차단되지 않고, (c) @supabase/ssr이 요구하는 쿠키 기반 세션 갱신이 요청마다 일어나지 않아 세션 만료 시 조용히 로그아웃된다. src/proxy.ts로 이동하고 export function proxy로 바꿔야 하며, 빌드 로그로 로드 여부를 확인해야 한다.

#3 RLS 재귀 정책과 미커밋 스키마. CLAUDE.md의 profiles admin 정책은 exists (select 1 from public.profiles where id = auth.uid() and role = 'admin')을 profiles 자신의 정책 안에서 평가한다. 이 서브쿼리는 다시 profiles RLS를 호출하므로 PostgreSQL이 재귀를 감지해 모든 profiles 조회가 실패한다. 미들웨어/콜백/레이아웃이 전부 profiles.role을 읽기 때문에 이 정책이 실제 DB에 적용되어 있으면 로그인 이후 전 경로가 깨진다. security definer 함수(public.is_admin())로 역할을 판정하거나 auth.jwt()의 app_metadata를 쓰는 방식으로 바꾸고, classes/subjects/attendance 정책과 handle_new_user 트리거를 포함한 마이그레이션 파일을 저장소에 커밋해야 한다. 현재는 "RLS가 걸려 있다"는 PRD 주장을 코드로 확인할 방법이 없다.

#4 .gitignore 삭제와 이중 저장소. HEAD의 .gitignore는 .env*.local, .next/, node_modules/를 제외하고 있었으나 작업 트리에서 삭제되었다. 이 상태에서 git add -A를 하면 Supabase 키가 들어 있는 .env.local이 커밋된다. 또한 상위 platform 저장소가 이 폴더의 소스 26개를 이미 추적하고 있어 두 저장소의 이력이 어긋난다(상위에는 삭제된 파일이 처음부터 없음). 즉시 .gitignore를 복원하고, 상위 저장소에서 서브모듈 또는 단일 저장소 중 하나로 정리해야 한다.

4. 효율성 개선 제안

- 구조/아키텍처
  - HEAD 기능을 복원한 뒤 src/lib/actions.ts를 도메인별(actions/classes.ts, actions/attendance.ts)로 나누고 각 액션 첫 줄에서 requireRole('admin') 헬퍼를 호출해 서버 측 인가를 고정한다. 현재 HEAD 액션들은 인가 검사 없이 RLS만 의존한다.
  - src/proxy.ts로 가드를 이전하고, admin/layout.tsx·student/layout.tsx에서도 role을 재확인해 2중 방어로 만든다.
  - 프로필 없는 사용자 처리: handle_new_user 트리거로 가입 즉시 profiles 생성(기본 student), 그래도 없으면 미들웨어가 /login?error=no_profile로 보내고 루프를 막는다.
- 성능
  - 대시보드 집계는 attendance 전체 행을 가져와 JS로 세는 대신 rpc('dashboard_stats', {date}) 하나로 처리.
  - RealtimeFeed는 payload(INSERT/UPDATE)로 피드 상단에 1건을 끼워 넣고, 조인 이름이 필요한 경우만 단건 조회.
  - 출석 보드(HEAD 버전)는 channel 필터를 class_id로 두고 있어 좋다. 과목 필터까지 포함하면 불필요한 이벤트가 줄어든다.
- 개발 생산성
  - supabase gen types로 Database 타입을 만들고 createServerClient<Database> 적용. (r: any) 제거.
  - Hospital/ERP와 같은 날짜 UTC 버그가 있으므로 세 프로젝트 공통 kstDateRange() 유틸을 공유 패키지로 뽑는다.
  - package.json: lint를 eslint .로, TypeScript/Node 타입 메이저를 Next 16 권장 범위로 고정.
- 운영/배포
  - supabase/migrations/0001_init.sql(테이블, RLS, 트리거, attendance unique(student_id, subject_id, date))과 seed.sql 커밋. Realtime publication에 attendance 추가 명시.
  - .gitignore 복원, .env.example 추가, 상위 저장소 추적 정리.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상규모	권장 모델	선정 이유
1	작업 트리 정리: 현재 수정분을 stash -> HEAD의 삭제된 10개 파일 복원 -> 미들웨어/대시보드 개선분 재적용 -> .gitignore 복원. 상위 platform 저장소와 nested git 중 하나로 통일	P0	S	Opus 5.5	git 이력 판단이 필요한 중간 난이도 작업, 코드 변경은 적음
2	middleware.ts -> src/proxy.ts 이전, role undefined 시 루프 방지(/login?error=no_profile), 빌드 로그로 로드 확인	P0	S	Fable 5.1	인증 가드 로직. 실수 시 전체 접근제어 무력화
3	supabase/migrations/0001_init.sql: 4개 테이블, is_admin() security definer 함수 기반 RLS(재귀 제거), handle_new_user 트리거, attendance unique 제약, Realtime publication	P0	M	Fable 5.1	보안·데이터 모델 핵심

#	작업	우선순위	예상 규모	권장 모델	선정 이유
4	admin/layout.tsx·student/layout.tsx에서 role 재검증 + redirect, 로그아웃 버튼 추가(Sidebar 하단)	P0	S	Opus 5.5	명확한 보안 보완 + 소규모 UI
5	복원된 `lib/actions.ts`에 `requireRole('admin')` 인가 헬퍼 적용, 입력 검증(zod), 에러를 Toast로 전달하는 반환 타입 정의	P1	M	Opus 5.5	2~5개 파일 리팩토링
6	KST 날짜 유틸 도입 후 대시보드/출석 보드/학생 대시보드 적용, 출석률 분모 통일(기록 수 기준 또는 "체크 대상 학생 수" 기준으로 PRD와 합의)	P1	S	Opus 5.5	집계 정의 정리
7	`supabase gen types` 적용, `any` 제거, join 타입 정의	P1	S	Sonnet 5.5	타입 오류 정리
8	학생 등록/초대 플로우(관리자가 이메일 초대 -> profiles에 class_id 배정) 구현 — PRD F-CLASS-02	P1	M	Opus 5.5	Supabase Auth Admin API 연동이 필요한 중간 규모 기능
9	`RealtimeFeed` payload 기반 부분 갱신, 대시보드 집계 RPC화	P2	S	Opus 5.5	성능/상태관리 정리
10	반응형 레이아웃(사이드바 햄버거, grid 브레이크포인트), lucide-react 아이콘으로 이모지 교체	P2	M	Sonnet 5.5	UI/스타일 작업
11	`package.json` 정리(`lint` 스크립트, TS/Node 타입 버전), .env.example, README 작성	P2	S	Sonnet 5.5	설정/문서
12	실시간 출석 변동 추이 그래프(F-DASH-01) — 시간대별 집계 뷰 + 경량 SVG 차트	P2	M	Opus 5.5	정의된 기능 구현

6. 종합 평가

- 강점
  1. nested HEAD 기준으로 보면 출석 upsert + Realtime 구독, 학급/과목 CRUD 서버 액션 등 PRD 핵심이 이미 한 번 구현된 적이 있어 복원 비용이 낮다.
  2. 역할 2개로 단순하고, 미들웨어 로직 자체(역할별 차단/리다이렉트)는 올바르게 작성되어 있어 위치만 옮기면 된다.
  3. Toast/Card/Badge 등 최소 UI 컴포넌트가 분리되어 있고 Design_UI.md의 색상 토큰이 코드에 반영되어 있다.
- 약점
  1. 작업 트리가 커밋보다 퇴행한 상태이고 `.gitignore`까지 삭제되어 저장소 관리가 무너져 있다.
  2. 미들웨어 미로드 + 레이아웃 역할 미검증 + RLS 미커밋으로 접근제어가 실질적으로 0이다.
  3. 스키마/마이그레이션이 저장소에 없어 DB 상태를 재현할 수 없고, 가입 시 프로필 생성 경로가 없어 신규 사용자가 루프에 빠진다.
- 모델별 비율: Fable 5.1 17% (2건) / Opus 5.5 58% (7건) / Sonnet 5.5 25% (3건)
- 권장 진행 순서
  1. 1단계(복구): 작업 1 -> 2 -> 3 -> 4. 삭제 파일 복원, proxy 이전, 마이그레이션 커밋, 레이아웃 방어선과 로그아웃.
  2. 2단계(견고화): 작업 5 -> 6 -> 7 -> 8. 서버 액션 인가, 날짜/집계 정합성, 타입, 학생 배정 플로우.
  3. 3단계(완성도): 작업 9 -> 10 -> 11 -> 12.
- 주 담당 모델: **Opus 5.5**. 코드 규모가 작고(복원 후 약 30개 파일) 역할이 2개뿐이어서 아키텍처 재설계보다는 "이미 있는 기능을 복원하고 올바른 위치에 놓고 인가를 붙이는" 명확히 정의된 작업이 대부분이다. 단, proxy 이전과 RLS/트리거 마이그레이션(작업 2, 3)은 실수 비용이 크므로 Fable 5.1이 먼저 처리한 뒤 나머지를 Opus

5.5가 이어받는 구성이 비용 대비 안전하다.